



PHP best practices 20m

As with other coding languages, there is a set of best practices you should demonstrate when developing your web applications. Like all technology, people are always finding new and better ways to do things and best practices are often improved.

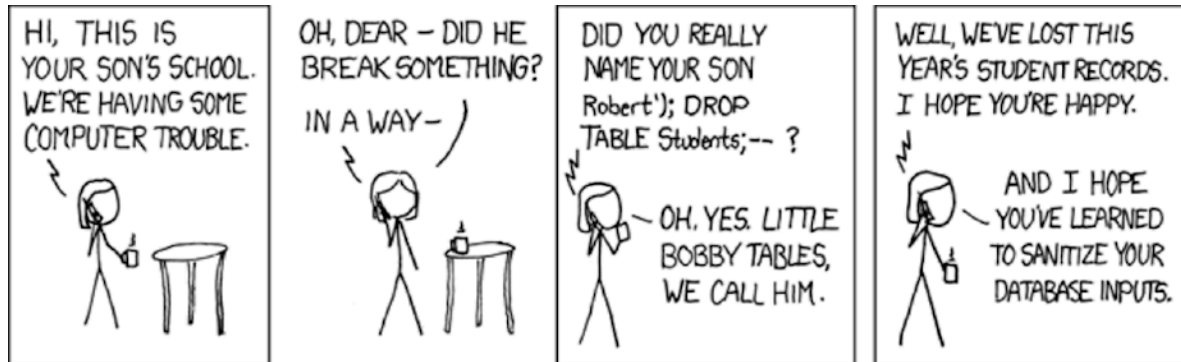
Once you enter the industry, you should keep up to date with the best practices of PHP to make sure you remain competitive. There is a set of codified standards for PHP development that have been created by the PHP development community over the years. They are important for two main reasons: security and maintainability.

Security

PHP is susceptible to malicious attacks, and so we want to follow best practices to secure our code. The core premise is simple: sanitise ALL data produced client-side. This is mostly for HTML forms that users can enter data into, including the image upload we will create later. The comic strip below shows us why. This principle also applies to the likes of AJAX, RESTful services and others.



[Skip to main content](#)



© © Randall Munroe



This is a well-known comic strip called 'Exploits of a Mom' (Munroe, 2018). The mother of this student tried to run an exploit on their student profile form. If we aren't careful, users can insert SQL into a form, and if this is then put into a query, the query can execute. In this case, if the user put in the following as a name:

```
Robert'); DROP TABLE Students;
```

then, when we update the user, we may have this SQL execute:

```
UPDATE Students  
  
SET `first_name` = Robert'); DROP TABLE Students;  
  
WHERE `id` = 1234;
```

Skip to main content
By the time the SQL runs:



```
DROP TABLE Students;
```

The table is deleted, and the school loses all the student records in their database. This is an SQL injection, which is a common exploit. So, we will be following best practice here and sanitising all data.

Maintainability

Developers have estimated that the average line of code may be read 10 times more often than it is written. For this reason, a collaborative group termed the Framework Interoperability Group (PHP-FIG) has established PHP coding conventions which should be followed. Two standards we will use are PSR-12 and PSR-4 (PSR stands for PHP Standard Recommendation).

PSR-4

PSR-4 describes a specification for autoloading classes from file paths.

Autoloading is the process of automatically loading PHP classes without explicitly loading them with `require()`, `require_once()` and `include()`.

It is fully interoperable and can be used in addition to any other autoloading specification, including PSR-0. This PSR also describes where to place files that will be autoloaded according to the specification.

PSR-12

PSR-12 replaced PSR-2 in 2019 and requires adherence to PSR-1. Most of the standards remain unchanged. This is primarily a way of establishing systematic coding, which means any PHP class, function or method is clear, clean and easily read and understood. You may like to bookmark the PSR-12 coding style guide. To summarise:

- Code **MUST** follow a 'coding style guide' PSR (PSR-1).
- Code **MUST** use four spaces for indenting, not tabs.
- There **MUST NOT** be a hard limit on line length. The soft limit **MUST** be 120

characters, and lines **SHOULD** be 80 characters or less.

Skip to main content

- There **MUST** be one blank line after the namespace declaration, and there **MUST** be one blank line after the block of use declarations.
- Opening braces for classes **MUST** go on the next line, and closing braces **MUST** go on the next line after the body.
- Visibility **MUST** be declared on all properties and methods; abstract and final **MUST** be declared before the visibility; static **MUST** be declared after the visibility.
- Control structure keywords **MUST** have one space after them; method and function calls **MUST NOT**.
- Opening braces for control structures **MUST** go on the same line, and closing braces **MUST** go on the next line after the body.
- Opening parentheses for control structures **MUST NOT** have a space after them, and closing parentheses for control structures **MUST NOT** have a space before.



Thoughts on this page?

